Superposition or Sampling in Disguise? What Soft Thinking Stores, Feeds and Uses

Hassan Nafees[1]

[1]Department of Computer Science, FAST National University of Computer and Emerging Sciences, Karachi, Pakistan

k230769@nu.edu.pk

***Abstract*—**Soft Thinking feeds a reasoning model blends of token embeddings instead of single tokens, in the hope that the model can carry several candidate continuations in superposition. Recent work adds Gumbel noise to the blend weights to avoid greedy collapse. With that noise, the top token of each blend is distributed exactly as a sampled token, so Gumbel Soft Thinking is ordinary sampling plus a residual weight on the other candidates. We test superposition at the level of single tokens in DeepSeek-R1-Distill-Qwen-1.5B on GSM8K: does the residual survive the layers, persist in the cache, and change what the model does? Exact 50/50 blends of two candidates, a forced superposition built at the input, collapse gradually through the layers. Information about the runner-up persists in the keys and values of layers 0 to 7 and shifts the projected next-token readout of later text toward it (+0.44, 95% CI [0.35, 0.53], number vs number forks); on the model's own path the shift is not detectable. In real runs the residual is small (median 0.12 at forks), and the top token changes as often as under sampling (0.45 vs 0.45). A coupled ablation with shared noise shows that the model reads the residual: it moves next-token predictions a median 3.9 times as much as a random residual of the same size (95% CI [3.1, 5.0]). Accuracy does not change detectably (88 vs 86 of 100; paired difference +2 points, 95% CI [−5, +9]), and the real and random residuals change sampling's final answer on largely the same problems (10 shared, 6 for each alone). Token-level superposition is therefore present in the input and read by the model, but we detect no use of it in the answers of this model; whether whole reasoning paths are carried in superposition is not tested here. The main predictions were committed before the runs, and every number from our experiments is recomputed from saved outputs.

1. Introduction

A reasoning model that writes out its chain of thought commits to one token at every step [1]. The other candidates at that step are discarded. Latent reasoning methods relax this commitment by feeding the model continuous states instead of tokens [2, 3]. A common motivation is *superposition*: a continuous state might hold several candidate paths at once and let the model explore them in parallel. In theory this can help, since a two-layer transformer with continuous thoughts can solve graph reachability by keeping several search frontiers in one state [4]. Soft Thinking [3] was proposed with the same intuition. It needs no training: at each thinking step it feeds the probability-weighted average of the top candidates' embeddings in place of one sampled token.

Whether off-the-shelf models use these blends is contested. Wu et al. [5] find that vanilla Soft Thinking is dominated by its top token and drifts into greedy, repetitive decoding, which they call the Greedy Pitfall. They add Dirichlet or Gumbel noise to the blend weights and report small accuracy gains over matched sampling. Rizvi-Martel et al. [6] find no sign that vanilla Soft Thinking reasons in superposition, and Backour [7] shows that pretrained models collapse injected mixtures of embeddings through depth.

Gumbel noise also changes what the superposition question asks. By the Gumbel-max trick [8, 9], the top token of a Gumbel blend is distributed exactly as a sampled token. Gumbel Soft Thinking therefore feeds the token that sampling would feed, plus a *residual*: the blend weight that sits on the other candidates (Fig. 1). If the model ignores the residual, Gumbel Soft Thinking is sampling in disguise. If the model carries both candidates forward and uses them, it reasons in genuine superposition. A third outcome lies between the two: the model may read the residual without its answers depending on it. Our research question is which of these holds:

> *Does Gumbel Soft Thinking reason in superposition, or is it sampling in disguise?*

We answer it at the level of single tokens. Every blend in this paper, whether an exact 50/50 blend or one fed by Gumbel Soft Thinking, is a *forced* superposition, built at the input, as opposed to a *learned* one that emerges from training [6]. We ask whether one token position carries two candidates and whether that changes what the model does. Whether whole reasoning paths are carried in superposition over many steps, as in the graph-search construction of Zhu et al. [4], is a different question that we do not test.

We split the question into four measurements (Fig. 2) on DeepSeek-R1-Distill-Qwen-1.5B [10, 11] solving GSM8K problems [12]. Q1 asks whether a blend of two candidates survives the layers. Q2 asks whether information about the losing candidate, the runner-up, persists in the key-value cache and shifts what later text predicts. Q3 asks what blends the noise actually feeds in real runs. Q4 asks whether the residual changes the model's next-token predictions and its final answers. We committed numeric predictions for the main tests before running them.

Our evidence supports three statements about token-level superposition in this model. First, the top token of Gumbel Soft Thinking is a sampled token. Second, the residual is read: it shifts the model's next-token predictions far more than a random residual of the same size. Third, no change in accuracy is detected on 100 problems, with a paired interval that still allows a few points either way. The superposition is present in the input and read, but we find no evidence that it is used. Our contributions are as follows.

- **Gumbel Soft Thinking is sampling plus a residual.** We make the Gumbel-max consequence explicit and confirm it in logged runs: at forks the top token changes at a rate of 0.45, the same as under sampling (0.45). Real forks feed lopsided blends, near one token at 47% of forks, with a median residual of 0.12. Dirichlet noise does not even match sampling: its top token leans toward the model's first choice (Section 4.3).
- **The runner-up persists in the cache.** Blends collapse gradually through the layers, as Backour [7] found in other models. Yet information about the runner-up persists, mostly in the keys and values of layers 0 to 7, which close a pooled share of 0.71 to 0.91 of the readback gap (pre-registered); part of this is an echo of the input in the first layers. Later text that turns toward the runner-up shifts the projected next-token readout toward it (+0.44 for number vs number forks). On the model's own path the shift is not detectable (Sections 4.1 and 4.2).
- **The model reads the residual, but accuracy does not change detectably.** A coupled ablation runs sampling, Soft Thinking and a random-residual control in lockstep with shared noise. Soft Thinking

leaves sampling first on 59 problems against 24 for the control ($p = 0.0002$) and moves next-token predictions a median 3.9 times as much (95% CI [3.1, 5.0]). Accuracy does not change detectably (88 vs 86 of 100; paired difference $+2$ points, 95% CI $[-5, +9]$), and paired by problem, the real and random residuals change sampling's final answer on mostly the same problems (Section 4.4).

- **A checkable study.** Predictions were committed before the runs and the failed ones are reported as written (Appendix A). Every number from our experiments is recomputed from saved outputs by scripts in our repository (Appendix D).

2. Related work

Soft Thinking and noise. Zhang et al. [3] introduced Soft Thinking, a training-free method that feeds probability-weighted mixtures of token embeddings during thinking, and reported small accuracy gains and shorter generations on math and code benchmarks. Wu et al. [5] probed the method and found that models rely mainly on the top token of the blend, which produces the Greedy Pitfall; they call the models single-threaded reasoners. They proposed Stochastic Soft Thinking with Dirichlet or Gumbel-Softmax noise [13, 14] and motivated the Gumbel variant through the Gumbel-max trick and its top-k extension [15]. They did not isolate the weight off the top token in a controlled test. We make the consequence of the trick explicit, check it in logged runs, and isolate the residual with a coupled ablation, which refines their single-threaded account for the Gumbel variant. Noisy soft tokens are also used for training: Butt et al. [16] add noise to soft input embeddings for reinforcement-learning exploration, and Zheng et al. [17] inject Gumbel noise and use Gumbel-Softmax in policy optimisation. Our analysis concerns a fixed, untrained model at inference.

Is superposition used? Zhu et al. [4] prove that continuous thoughts can encode several search frontiers in one state, and Hao et al. [2] report breadth-first-search-like latent states in Coconut. Deng et al. [18] train latent reasoning to act as a superposition of reasoning paths. Rizvi-Martel et al. [6] separate *forced* superposition, constructed at the input as in Soft Thinking, from *learned* superposition that emerges from training with latent thoughts. They test three regimes and find signs of superposition being used only in models trained from scratch; in the training-free regime, vanilla Soft Thinking looks much like discrete decoding under the logit lens. We study forced superposition in the training-free regime with noise, condition on forks where the model is uncertain, and go beyond the next-token readout to what persists in the key-value cache and to causal use. Like theirs, our tests are at the level of single positions, not of whole reasoning paths.

Collapse of mixtures. Backour [7] injects two-way mixtures of embeddings into pretrained models and projects each layer's state onto the line between the two pure states. Backour finds that trained models collapse mixtures far more than random-initialised controls and that softmax readout with autoregressive feedback amplifies small imbalances until one component dominates. Our snap score is his CALIB measure applied at natural forks of a reasoning model. We then ask what the collapse leaves behind, what blends the noise feeds, and whether the residual changes predictions and answers.

Causal analysis of latent reasoning. Activation patching localises information by swapping internal states between runs [19]; our key-value patching applies it to the cache entries of one position. For trained latent reasoners, Li et al. [20] intervene on latent steps of Coconut and CODI and report a gap between early output bias and late representational commitment. Zhang et al. [21] find that Coconut's latent tokens are insensitive to steering and support shortcuts, and Aswal et al. [22] show that decodable latent patterns also appear in controls and need not affect behaviour. Jin et al. [23] track latent-reasoning faithfulness across training. Following their advice, each of our causal tests has a matched control: a control token in Q2 and a random residual in Q4.

Two senses of superposition. In interpretability, superposition usually means that a network represents more features than it has dimensions [24]. We use the word only in the latent-reasoning sense above: several candidate continuations carried and used at once.

3. Setup

Model. All experiments use DeepSeek-R1-Distill-Qwen-1.5B [10, 11], a reasoning model with 28 layers, in fp32 on one NVIDIA T4 GPU. Every run loaded the same model revision (Appendix D). We name the four experiments by what they measure: the *accuracy comparison*, the *fork probe* (Sections 4.1 and 4.2), the *fed-blend log* (Section 4.3) and the *coupled ablation* (Section 4.4). Appendix D maps each to its script and saved output.

Decoding. We follow the public implementation of Soft Thinking [3]. At each thinking step the logits are divided by a temperature of 0.6 and filtered to at most 10 candidates (top-k 10, top-p 0.95, min-p 0.001), and the weights p_k are renormalised. Inside `<think>`, Soft Thinking feeds the blend $\sum_k w_k e_k$ of the candidates' input embeddings e_k instead of one token. Vanilla Soft Thinking uses $w = p$. Gumbel Soft Thinking uses $w = \mathrm{softmax}\big((\log p + g)/\tau\big)$ with independent standard Gumbel noise g and $\tau = 0.5$, and Dirichlet Soft Thinking draws $w \sim \mathrm{Dirichlet}(\gamma p)$ with $\gamma = 1.0$ [5]. The sampling arm is ordinary chain-of-thought decoding: it samples one token from the same filtered candidates and feeds its embedding. Thinking ends when the top token of the fed blend is `</think>` or end-of-sequence, at Cold Stop, or after 8,192 steps. We then feed a real `</think>` and decode the answer greedily for at most 512 tokens. The public implementation samples the answer at temperature 0.6; we decode greedily so that the answer is a deterministic function of the thinking trace. *Cold Stop*, adapted from Zhang et al. [3], ends thinking when the entropy of the pre-noise weights stays below 0.01 for 256 consecutive steps, which catches repetition loops. It applies to the Soft Thinking arms only in the accuracy comparison and to all three arms in the coupled ablation. Our settings differ from those of Wu et al. [5], who use top-k 30, a cap of 32,768 thinking tokens and no Cold Stop. Within each of our comparisons every arm uses the same settings, so the comparisons stay matched.

Task and problems. We use GSM8K test problems [12] with the prompt suffix "Please reason step by step, and put your final answer within `\boxed{}`." An answer is scored by the last number inside the last `\boxed{}`. Each experiment uses a fixed range of test items: the accuracy comparison items 0–29, the fork probe items 10–49, the fed-blend log items 10–29 and the coupled ablation items 50–149. The fed-blend log uses a subset of the fork probe's items and overlaps the accuracy comparison, so these are not independent samples. The coupled ablation uses problems that no other experiment touched. An earlier attempt on ProsQA was abandoned because most runs did not finish within the thinking budget, which is why we use GSM8K. An earlier, smaller test of answers at single forks was null; its saved output is incomplete, so we do not report it, and the coupled ablation replaces it. Earlier comparisons with the logit lens were withdrawn after we found two errors in them (the final normalisation was applied twice, and the lens was read at the forward pass that produced the blend instead of the one that consumed it).

Forks and blends. A *fork* is a thinking step at which the runner-up holds at least 25% of the pre-noise filtered weight. The *chosen token* is the top candidate, the *runner-up* is the second, and the *control token*

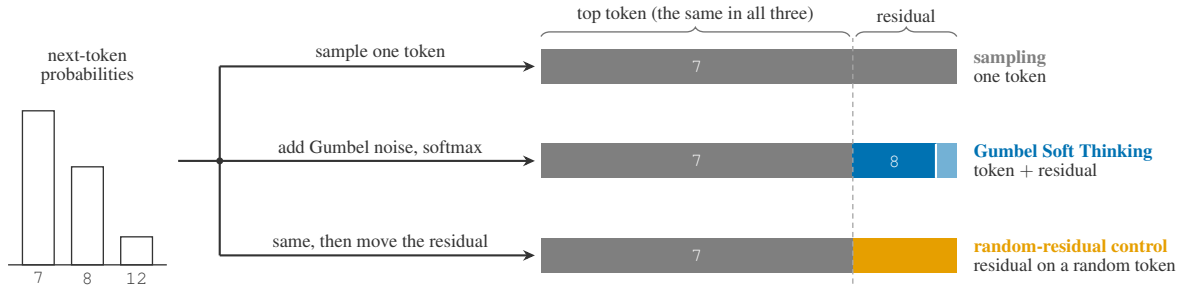


Figure 1. Gumbel Soft Thinking feeds the sampled token plus a residual. At each thinking step the model gives next-token probabilities. Sampling feeds one token. Gumbel Soft Thinking adds Gumbel noise and feeds the softmax-weighted mix of token embeddings. By the Gumbel-max trick, its top token is distributed exactly as the sampled one, and it is the same token when both use the same noise in the same context, as in our coupled ablation (Section 4.4). So all it adds is the residual (blue). The random-residual control keeps the top token and the residual's size but moves the residual to a random token (orange). Weights are illustrative.

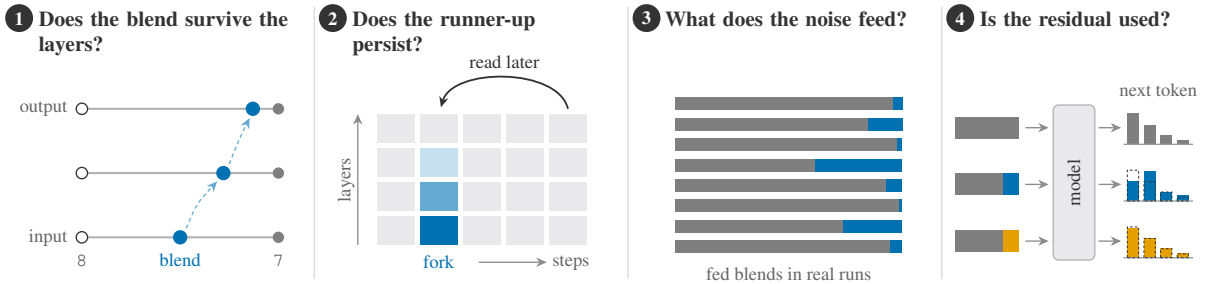


Figure 2. The four questions we test. (1) Does a blend of two candidates survive the layers, or snap to one of them (Fig. 3)? (2) Does information about the runner-up persist in the K,V cache and shift what later steps predict (Figs. 4 and 5)? (3) What blends does the noise actually feed in real runs (Fig. 6)? (4) Does the residual move the model's next-token predictions more than a random residual of the same size (Fig. 7, Table 1)? Drawings are schematic.

is the third. We classify a fork by whether the chosen token and the runner-up contain a digit: number vs number, number vs word, or word vs word. The fork probe takes its forks from vanilla and Gumbel Soft Thinking traces (seed 0) and finds 694 forks, 670 of them unique: 238 number vs number from 40 problems, 56 number vs word from 24 and 376 word vs word from 40. At each fork it feeds blends from the same cached prefix: the sweep $\alpha e_A + (1-\alpha)e_B$ (Section 4.1), the 50/50 blend of chosen token and runner-up, and the 50/50 control blend of chosen token and control token (Section 4.2). We call the blend weight that is not on the top token the *residual*, and the largest fed weight the *top weight*. The fed-blend log records the pre-noise and fed weights of real runs (Section 4.3). The coupled ablation runs sampling, Soft Thinking and the random-residual control in lockstep (Section 4.4). The random token is drawn once per step from the real tokens of the vocabulary, by the same keyed generator as the Gumbel noise, so all arms see the same draw.

Validation gates. Before the experiments, the code passed a set of checks, all of which are in the scripts in our repository. A hand-written greedy loop reproduces `model.generate()`. The final hidden state reproduces the live logits under exactly one normalisation convention. In vanilla runs the top token of the fed blend equals the raw argmax at every step. At layer 0 the sweep position equals the mixing weight (Section 4.1). Forcing a text in one forward pass matches generating it token by token. Patching the fork position in all 28 layers reproduces the donor cache. In the coupled ablation, two copies of the sampling arm agree for 64 steps. None of the reported experiments uses the logit lens.

Pre-registration. Before running the fork probe we committed six predictions with numeric thresholds (P1–P6) to our repository. The thresholds were set on a 10-item pilot and then tested on 40 held-out items. Predictions for the fed-blend log and the coupled ablation were committed the same way before those runs. Git dates come from the local machine, so they show the order of commits, not an external timestamp. Appendix A lists every prediction as written, with its outcome, including the ones that failed.

Statistics. Forks and steps from the same problem are not independent, so every CI on fork- or step-level quantities is a 95% percentile bootstrap that resamples problems, 2,000 times, with all forks of a drawn problem kept together. Accuracies have Wilson 95% intervals [25], except where runs share problems: in the accuracy comparison, two seeds run on the same 30 problems, so those intervals are bootstrapped over problems. Paired accuracy comparisons use the exact two-sided McNemar test [26] on problems where exactly one arm is correct, and the paired difference in accuracy has Newcombe's hybrid-score 95% interval [27]. The paired order of divergence in the coupled ablation uses the exact two-sided sign test, ignoring ties. The coupled ablation compares next-token distributions by the KL divergence of each arm's distribution from sampling's, computed over the full vocabulary at the sampling temperature. All runs use seed 0 unless stated; the accuracy comparison adds seed 1 for sampling and Gumbel Soft Thinking.

4. Results

We take the four questions of Fig. 2 in turn. Sections 4.1 and 4.2 use the fork probe, Section 4.3 the fed-blend log, and Section 4.4 the coupled ablation and the accuracy comparison.

4.1. Q1: Does a blend of two candidates survive the layers?

If Soft Thinking carried two candidates forward, a blend of them would have to survive the forward pass. We measure this with the method of Backour [7]. At each fork we feed $\alpha e(A) + (1-\alpha)e(B)$ for $\alpha = 0, 0.1, \ldots, 1$ from the same cached prefix, where A is the chosen token and B the runner-up. Let $h_\ell(\alpha)$ be the hidden state at layer ℓ of the fork position when this blend is fed, so that $h_\ell(1)$ belongs to the pure chosen token and $h_\ell(0)$ to the pure runner-up. We place each blend on the line

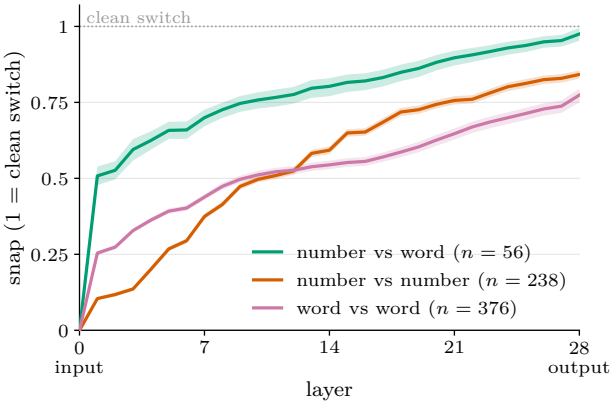


Figure 3. Blends collapse gradually through the layers. At each fork we feed a blend of the chosen token and the runner-up and measure how sharply each layer resolves it (snap, (2): 0 = carried linearly, 1 = clean switch). Snap is 0 at the input by construction and reaches 0.98 (number vs word), 0.84 (number vs number) and 0.77 (word vs word) at the output. Number vs word snaps hardest at every layer from 1 to 28; number vs number starts slowest and passes word vs word at layer 13. Bands: 95% bootstrap CIs over problems. 670 unique forks from 40 GSM8K problems (items 10–49). Inputs are sweeps between two candidates at forks, not the lopsided blends real runs feed (Section 4.3).

between the two pure states and compare its position with the mixing weight:

$$c_\ell(\alpha) = \frac{\big(h_\ell(\alpha) - h_\ell(0)\big) \cdot \big(h_\ell(1) - h_\ell(0)\big)}{\|h_\ell(1) - h_\ell(0)\|^2}, \tag{1}$$

$$\text{snap}_\ell = \frac{\sum_\alpha |c_\ell(\alpha) - \alpha|}{\sum_\alpha |\mathbb{1}[\alpha > 0.5] - \alpha|}, \tag{2}$$

with both sums over the 11 values of α. The denominator is the same quantity for a perfect switch at $\alpha = 0.5$, so a snap of 0 means the blend is carried linearly and 1 means a clean switch to one candidate. On our grid, snap is 1 minus the CALIB score of Backour [7]. At the input the position equals the mixing weight, as it must (largest error 1.5×10^{-7}).

Figure 3 shows the result for the 670 unique forks of the fork probe. Snap rises from 0 at the input to between 0.37 and 0.70 at layer 7, and to between 0.77 and 0.98 at the output. How far the two candidates differ in kind matters at the output. Number vs word snaps hardest (0.98, 95% CI [0.95, 1.00]), then number vs number (0.84 [0.83, 0.86]) and word vs word (0.77 [0.75, 0.79]). This order holds at the output, not at every layer. Number vs word is highest at every layer from 1 to 28, but number vs number starts slowest: it is below word vs word at layers 1 to 12 and above it from layer 13 to the output. At an exact 50/50 input, the next token is the pure chosen token's or the pure runner-up's at 95% (number vs number), 98% (number vs word) and 88% (word vs word) of the forks where the two differ. The blended states also stay close to the line between the two pure states: for number vs number at the output, the distance off the line is 0.28 of the distance between them. The noise that produced the trace does not change how a blend is processed (number vs number at the output: 0.85 for forks from vanilla traces, 0.84 from Gumbel traces). Both pre-registered predictions on collapse were met (P1, P2; Appendix A).

Answer. Only partly. By the output a 50/50 blend is mostly resolved, and the next token follows one of the two candidates at 88 to 98% of forks. The collapse is gradual, though: at layer 7 the blend is still far from a clean switch. Whether information about the runner-up is lost, or only absent from the next-token prediction, is the next question.

4.2. Q2: Does the runner-up persist in the cache?

The next-token prediction is not the only place a candidate can survive. Later steps attend to the keys and values (K,V) of the fork position, so information about the runner-up could persist there and shift what later text predicts. We test this at the forks where the model followed the chosen token at the 50/50 input: 96 number vs number forks from 35 problems and 139 word vs word forks from 40 problems, plus 34 number vs word forks.

Readback. For each fork we build two caches: one with the 50/50 blend of the chosen token and the runner-up at the fork position, and a control cache with a 50/50 blend of the chosen token and a control token. The control token is the third-ranked pre-noise candidate, so it also fits the context but holds no runner-up. We then force a 64-token continuation into both caches. It is either the runner-up's own continuation, or the model's own continuation of the chosen token. At each forced position we take the model's next-token distribution p at the sampling temperature and place it on the line from the distribution p_B after the pure runner-up to the distribution p_A after the pure chosen token: $c = (p - p_B) \cdot (p_A - p_B)/\|p_A - p_B\|^2$. A score of 1 means "as if the chosen token had been written" and 0 "as if the runner-up had been written". The runner-up readback is the control score minus the blend score. It is above 0 when the blend cache makes the model read the text more as if it had written the runner-up. Positions where p_A and p_B differ by less than 0.05 in total variation are skipped.

With the runner-up's text, the blend cache shifts the readout toward the runner-up (Fig. 4a). The readback is $+0.44$ for number vs number (95% CI [0.35, 0.53]), $+0.33$ for word vs word ([0.27, 0.40]) and $+0.11$ for number vs word ([0.03, 0.19]). On number vs number forks the blend cache moves the projected readout 0.44 of the way from "as if the chosen token had been written" toward "as if the runner-up had been written". This is a shift in a projected next-token readout. It shows that information about the runner-up persists in the cache and reaches later predictions; it does not show that the runner-up is held as a separate candidate that the model could retrieve whole. For number vs word the trace is mostly erased, but not completely. The effect lasts: for number vs number and word vs word the CI stays above 0 in every position bin up to 64 tokens (Fig. 4b), and without binning the mean is above 0 at every single position from 1 to 16 (Fig. 8). With the model's own text the picture is different (Fig. 4c). The readback is -0.01 [-0.06, 0.04] for number vs number, $+0.02$ [-0.02, 0.07] for word vs word and $+0.06$ [-0.00, 0.19] for number vs word, and the CI includes 0 in every position bin for number vs number and word vs word. On the path the model actually takes, no shift toward the runner-up is detectable. These results met pre-registered predictions P3 and P4 (Appendix A).

Where the runner-up's trace sits. To locate it, we swap only the fork position's keys and values between the blend cache and the control cache, at one layer or one block of layers, and force the runner-up's text as before. *Remove* puts the control's K,V into the blend cache and asks whether these layers are needed. *Insert* puts the blend's K,V into the control cache and asks whether they are enough. We report the pooled share of the blend–control readback gap that the swap closes, summed over forks.

The trace sits mostly in layers 0 to 7 (Fig. 5a,b). These layers close a pooled share of 0.77 (remove) and 0.71 (insert) of the gap for number vs number, and 0.77 and 0.91 for word vs word. Layers 12 to 27 close 0.12 and 0.26, and 0.08 and 0.14. This split was pre-registered (P5, P6; the smallest lower bound of the difference is 0.31). By blocks of four layers, layers 0–3 carry most of it (number vs number 0.58 / 0.55, word vs word 0.66 / 0.75 for remove / insert), layers 4–7 add 0.19 / 0.16 and 0.10 / 0.16, and every block from layer 16 on closes at most 0.04

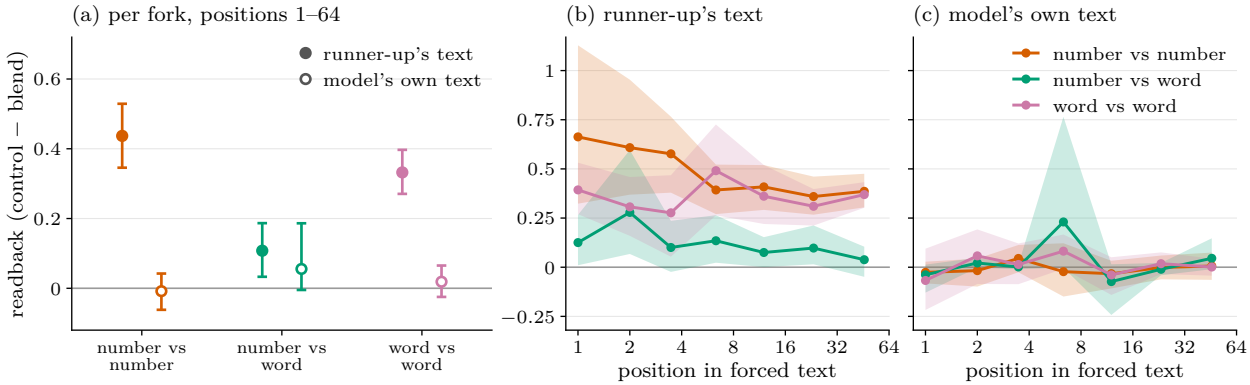


Figure 4. Information about the runner-up persists in the cache and shifts later predictions toward it, but not on the model's own path. At forks where the model followed the chosen token, we force a 64-token continuation into the cache built from the 50/50 blend and into a control cache (chosen token blended 50/50 with the third-ranked candidate), and score the next-token predictions from 0 (as if the runner-up had been written) to 1 (as if the chosen token had). Readback = control minus blend; above 0 means the blend cache shifts the projected readout toward the runner-up. (a) Per fork: with the runner-up's text, +0.44 (number vs number), +0.11 (number vs word) and +0.33 (word vs word); with the model's own text, −0.01 to +0.06, with every CI including 0. (b) With the runner-up's text, the readback stays above 0 in every position bin up to 64 tokens for number vs number and word vs word. (c) With the model's own text, the CI includes 0 in every bin for number vs number and word vs word. Positions are binned on a log scale; Fig. 8 shows them unbinned. Bars and bands: 95% bootstrap CIs over problems. n = 96, 34 and 139 forks. Exact 50/50 blends are the most favourable case for the runner-up, so these values are likely an upper bound for real forks, if readback grows with the residual (Section 4.3).

for number vs number. For word vs word every block from layer 8 on closes at most 0.07. Across blocks the effects add up: the number vs number blocks sum to 0.95 (remove) and 1.06 (insert).

Two findings below this level are exploratory: we found them after looking at the data. First, for number vs number the early effect sits in layers 0 to 2 (Fig. 5c). Single layers close 0.38 / 0.16 (layer 0), 0.21 / 0.07 (layer 1) and 0.20 / 0.05 (layer 2). Outside layers 0–3 only layers 7 (0.13 / 0.12) and 13 (0.08 / 0.13) stand out; every other layer closes at most 0.05. All five survive a Bonferroni correction over the 56 single-layer tests, although the insert effects of layers 1 and 2 only just do (corrected lower bound 0.01; Table 4). Second, inside layers 0–3 single layers do not add up to their block. Removing one layer at a time sums to 0.84 against 0.58 for the block, and inserting one at a time sums to 0.32 against 0.55. One reading is that the trace is held redundantly across layers 0–2: removing any one of them breaks the readback, but no single one carries it. Another is that the readback score saturates, since it is a projection that can pass 1. Our data do not separate the two.

The pooled shares are not driven by one problem or a few forks. Dropping any one problem changes a pooled share by at most 0.04, and keeping only forks with a gap of at least 0.1 changes none by more than 0.03 (Table 3). The 10 forks with the largest gap carry 0.27 (number vs number) and 0.24 (word vs word) of the total gap. Layer 0 is the exception to a uniform picture (Fig. 9). Its pooled remove share is 0.38, but the median fork's share is 0.12 (insert: 0.16 against 0.02), with quartiles of 0.01, 0.12 and 0.68. Layer 0 matters a lot at some forks and little at about half of them.

Part of the early-layer share is expected by construction. At layer 0 the keys and values of the fork position depend only on the fed embedding, which contains the runner-up, so the runner-up is present there as an echo of the input. There is also a small processed trace: for number vs number, layers 7 and 13 stand out (exploratory, as above).

Answer. Yes, as a trace. Information about the runner-up persists, mostly in the keys and values of layers 0 to 7 and partly as an echo of the input, and text that turns toward the runner-up shifts the projected readout toward it for the whole 64-token continuation. On the model's own path no such shift is detectable. All of this is measured at exact 50/50 blends, the most favourable case for the runner-up.

4.3. Q3: What blends does the noise actually feed?

Sections 4.1 and 4.2 feed exact 50/50 blends. Real runs feed whatever blend the noise produces, so we record it. The fed-blend log replays ordinary vanilla, Gumbel and Dirichlet runs on 20 problems and logs the pre-noise and the fed weights at every thinking step. Forks make up 9.2% (vanilla), 13.0% (Gumbel) and 13.3% (Dirichlet) of thinking steps. There are 3,774, 5,761 and 4,978 forks; each arm is its own run, so the forks differ.

The top token of Gumbel Soft Thinking is a sampled token. Gumbel noise makes the top token of the fed blend a sample from the filtered distribution. This is the Gumbel-max trick [8, 9]: the token with the largest perturbed log-probability is distributed exactly as a draw from the distribution, and the softmax at temperature τ does not change which token is largest. Wu et al. [5] cite the trick to motivate Gumbel-Softmax; we check its consequence in the logged runs. At forks, Gumbel makes a token other than the pre-noise top token the winner at a rate of 0.45, and ordinary sampling would do so at 0.45 (one minus the pre-noise top weight). The difference is 0.00 (95% CI [−0.01, 0.02]). Binned by the sampling rate, Gumbel stays within 0.04 of sampling in every bin with at least 100 forks (Fig. 6b). Dirichlet noise has no such guarantee, and it does not match sampling: it flips less often, by −0.03 [−0.04, −0.02], and is below sampling in all 10 bins. Its winner leans toward the pre-noise top token. So Dirichlet Soft Thinking is not sampling in disguise in the exact sense that Gumbel Soft Thinking is.

The residual is real but small. Before noise, the median fork has a residual of 0.44: the model splits its weight. Vanilla feeds these weights unchanged, so every vanilla fork is balanced, with a top weight of at most 0.75 by the fork rule (Fig. 6a). Noise makes the fed blends lopsided. Gumbel Soft Thinking feeds a blend near one token (top weight at least 0.9) at 47% of forks, a lopsided blend (0.75 to 0.9) at 22% and a balanced one at 31%. The median fed blend has a top weight of 0.88 and a residual of 0.12. Dirichlet gives 37%, 25% and 38%, with a median residual of 0.17. At the median fork, almost all of the residual sits on one other token (0.99 of it for Gumbel and 1.00 for Dirichlet). Over all thinking steps, the fed blend is near one token at 85% (Gumbel) and 83% (Dirichlet) of steps. We had pre-registered that Gumbel would feed near one-token blends at 70% or more of forks and 90% or more of steps; both failed (Appendix A).

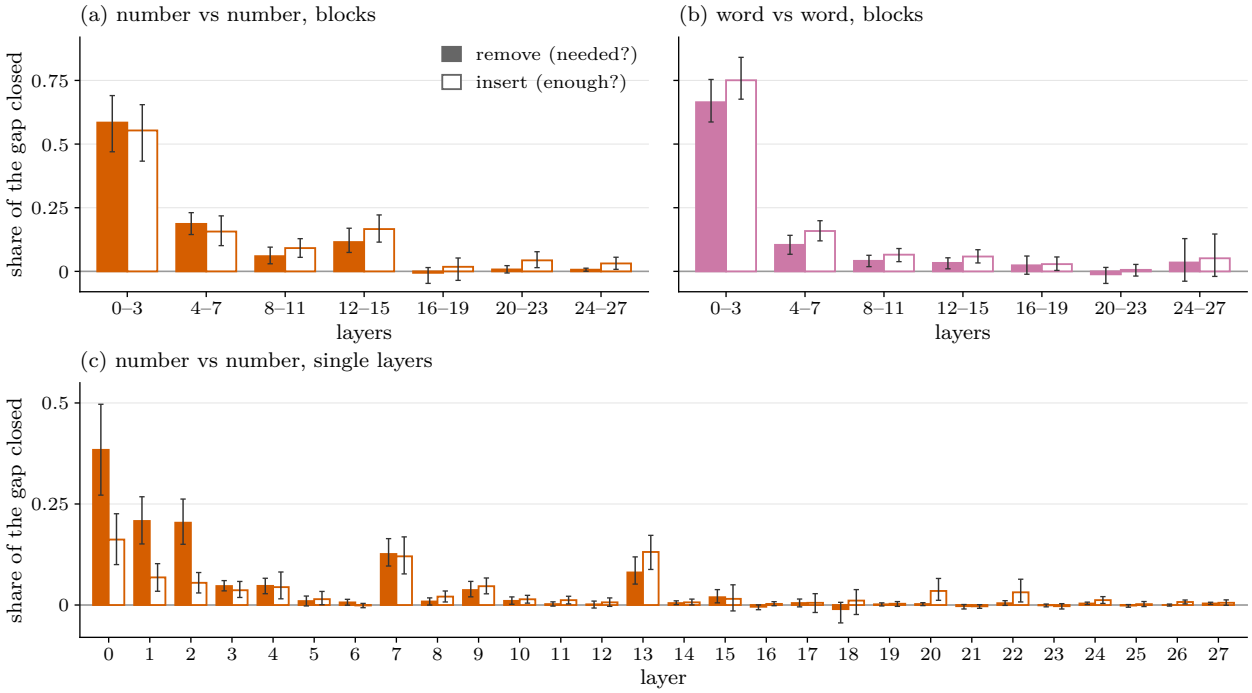


Figure 5. The cache entries that carry the runner-up's trace are mostly in the early layers. At each fork (50/50 blend; forks where the model followed the chosen token) we swap the keys and values of the fork position between the blend cache and the control cache at a set of layers, and report the pooled share of the blend–control readback gap this closes (summed over forks). Remove (filled): put control K,V into the blend cache (are these layers needed?). Insert (open): put blend K,V into the control cache (are they enough?). (a, b) Layers 0–7 close 0.77 / 0.71 (number vs number, remove / insert) and 0.77 / 0.91 (word vs word) of the gap; layers 12–27 close 0.12 / 0.26 and 0.08 / 0.14. (c) Number vs number, single layers: the early effect sits in layers 0–2 (remove 0.38, 0.21, 0.20). Outside layers 0–3, only layers 7 (0.13 / 0.12) and 13 (0.08 / 0.13) stand out, with CIs excluding 0; every other layer closes at most 0.05. Note the different y-axis in (c). The single-layer analysis was not pre-registered; Table 4 shows these layers survive a Bonferroni correction over all 56 single-layer tests. Table 3 shows the shares hold when any one problem, or forks with a small gap, are dropped; Fig. 9 shows the per-fork spread, which is wide for layer 0. Error bars: 95% bootstrap CIs over problems. 96 forks from 35 problems (number vs number) and 139 forks from 40 problems (word vs word). Number vs word is not shown: its gap (+0.11) is too small to localise (Table 5). All shares are measured at exact 50/50 blends, the most favourable case for the runner-up, so these shares are likely an upper bound for real forks.

Answer. Gumbel Soft Thinking feeds the sampled token plus a residual. At forks the residual is real but small: its median is 0.12, and at about half of forks the blend is near one token. The exact 50/50 blends of Sections 4.1 and 4.2 are the extreme case, so the storage and readback measured there are likely an upper bound on what a real fork leaves in the cache, if readback grows with the residual.

4.4. Q4: Does the model use the residual?

If the top token of Gumbel Soft Thinking is a sampled token, any difference from sampling must come from the residual. The coupled ablation isolates it. Three arms run in lockstep on 100 GSM8K problems never used before (items 50–149), with shared Gumbel noise keyed by seed, step and token id. *Sampling* feeds the top token alone (a one-hot input). *Soft Thinking* feeds the full Gumbel blend. The *random-residual control* keeps the top token at its blend weight and moves the residual onto a random vocabulary token. Because the noise is shared, the three arms pick the same top token at a step until their contexts differ, and they differ only in where the residual goes. The random rows have the same median norm as all real-token rows, and 0.994 of that of reasoning tokens (Appendix C), so the control carries a residual of the same size and a typical norm.

The model reads the residual. Soft Thinking leaves sampling sooner than the random-residual control (Fig. 7a). The first step at which its token differs from sampling's has a median of 35.5, against 66 for the control. Paired by problem, Soft Thinking leaves first on 59 problems, the control on 24, and both at the same step on 17 (sign test, $p = 0.0002$). To compare the arms before their texts differ, we average the KL divergence of each arm's next-token distribution from sampling's over the steps before either arm leaves sampling (a median of 31 steps per problem). Soft Thinking moves the predictions more than the control on 94 of 100 problems. The median per-problem ratio is 3.9 (95% CI [3.1, 5.0]; Fig. 7b). We chose the median after seeing that the ratio of the means, 5.7, is driven by a few problems: five problems hold 0.54 of Soft Thinking's total, and without them the ratio of means is 3.1. The pre-registered prediction E1 expected the opposite direction, that the residual would move predictions less than a random one (Appendix A). E1 was written for the KL averaged over each arm's own steps up to and including its first divergence. On that registered measure, Soft Thinking moves predictions less than the control on only 14 of 100 problems (mean KL 0.545 against 0.009). That average is dominated by the divergence step itself, and Soft Thinking diverges sooner, so we report the matched steps above, on which Soft Thinking moves predictions less on 6 of 100. Both measures reject E1 in the same direction.

Accuracy does not change detectably. Table 1 compares accuracy. In the coupled ablation, Soft Thinking solves 88 of 100 problems and sampling 86. The discordant pairs split 6 to 4 (McNemar $p = 0.754$), and the paired difference is +2 points with a 95% CI of [−5, +9]. The data are compatible with Gumbel Soft Thinking being up to 5 points worse or up to 9 points better than sampling on these problems, so the null is an absence of evidence at this size, not evidence of no effect. The random-residual control solves 82 (3 to 7 against sampling, $p = 0.344$;

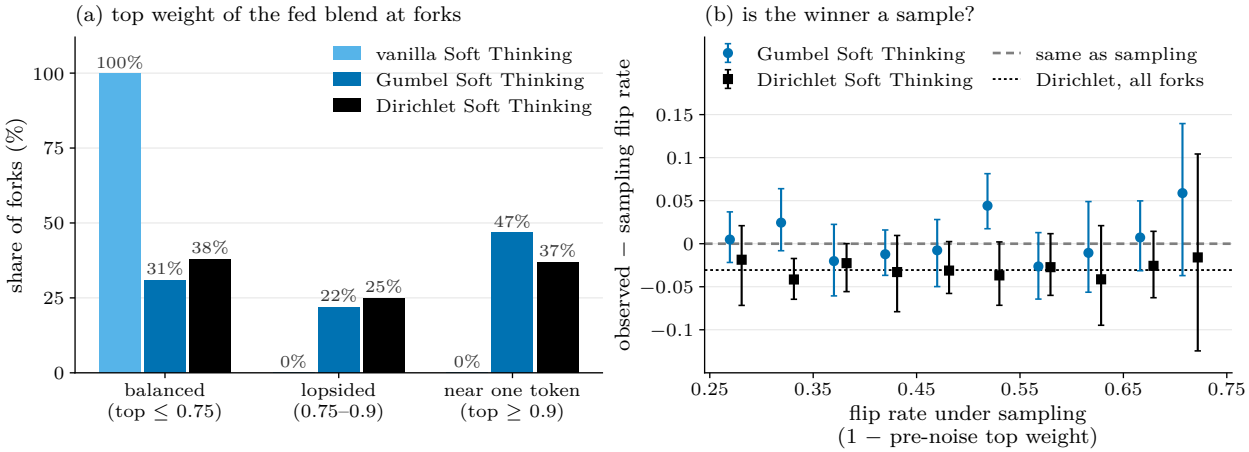


Figure 6. Noise turns real forks into lopsided blends, and Gumbel's winner is a sample. A fork is a thinking step where the pre-noise runner-up holds at least 25% of the weight (9.2% of steps for vanilla, 13.0% for Gumbel, 13.3% for Dirichlet). (a) Share of forks by the top weight of the blend actually fed. Vanilla feeds the pre-noise weights unchanged, so it shows what the model wanted: every fork is balanced (top weight at most 0.75, by the fork rule). Gumbel feeds a near one-token blend (top weight at least 0.9) at 47% of forks and Dirichlet at 37%. The median fed blend has a residual (weight off the top token) of 0.12 for Gumbel and 0.17 for Dirichlet, against 0.44 before noise. (b) How often noise makes a token other than the pre-noise top one the winner, minus how often ordinary sampling would (1 minus the pre-noise top weight), with forks binned by the sampling rate. Gumbel matches sampling: 0.00 [−0.01, 0.02] overall and within 0.04 in every bin with at least 100 forks. Dirichlet flips less often than sampling in every bin, −0.03 [−0.04, −0.02] overall (dashed line). Vanilla never flips (not shown). Error bars: 95% bootstrap CIs over problems. 3,774 (vanilla), 5,761 (Gumbel) and 4,978 (Dirichlet) forks from 20 GSM8K problems; each arm is its own run, so the forks differ.

Table 1. Neither Gumbel Soft Thinking nor the random-residual control differs detectably from sampling in accuracy, and the paired intervals still allow differences of several points either way. Accuracy comparison: sampling and Gumbel Soft Thinking at seeds 0 and 1 on the same 30 problems, so their 60 runs are clustered by problem and their CIs are 95% bootstrap intervals over problems (per seed: sampling 27 and 25 of 30, Gumbel 26 and 26); vanilla Soft Thinking has one seed and a Wilson interval. Its difference from sampling is paired on seed 0 (Newcombe interval); Gumbel's is the mean per-problem difference over both seeds, bootstrapped over problems. McNemar uses the seed 0 runs, the only seed vanilla has. Vanilla hit Cold Stop on 14 of 30 runs, and all 5 runs without an answer ended there; against Gumbel (seed 0) it is 1 vs 5, $p = 0.22$. Coupled ablation: seed 0; the three arms share their noise, and the random-residual control moves the residual to a random token. Accuracy has Wilson intervals and the paired difference a Newcombe hybrid-score interval. Its problems are not used in any other experiment. McNemar: exact two-sided test on the problems where exactly one of the pair is correct.

					McNemar vs sampling		
Arm	Runs	Correct	Accuracy [95% CI]	Difference vs sampling [95% CI]	Arm only	Sampling only	p
Accuracy comparison: GSM8K items 0–29							
Sampling	60	52	0.87 [0.75, 0.97]				
Vanilla Soft Thinking	30	22	0.73 [0.56, 0.86]	−0.17 [−0.33, −0.01]	0	5	0.06
Gumbel Soft Thinking	60	52	0.87 [0.73, 0.97]	0.00 [−0.10, +0.10]	1	2	1.00
Coupled ablation: GSM8K items 50–149, arms in lockstep							
Sampling	100	86	0.86 [0.78, 0.91]				
Gumbel Soft Thinking	100	88	0.88 [0.80, 0.93]	+0.02 [−0.05, +0.09]	6	4	0.75
Random-residual control	100	82	0.82 [0.73, 0.88]	−0.04 [−0.11, +0.03]	3	7	0.34

difference −4 points [−11, +3]). The pre-registered prediction E2 (no detectable accuracy difference, judged on the point estimate) was met.

Real against random residual, paired. Each residual arm reaches sampling's final answer on 84 problems, but equal counts do not show that the two arms change answers on the same problems, so we pair them directly. They change sampling's final answer together on 10 problems, Soft Thinking alone on 6 and the control alone on 6 (McNemar $p = 1.00$), and they reach the same final answer as each other on 85. Most answer changes therefore happen on problems that are fragile under any residual, and the real residual does not change more answers than a random one. Where the two arms differ in correctness, Soft Thinking is right on 9 problems and the control on 3 ($p = 0.146$; difference +6 points, 95% CI [−1, +13]). This is not significant, but it is the largest contrast in the study, and its interval leaves room for an effect of the residual's content of up to 13 points; part of it is the control's own deficit against sampling.

A second, smaller accuracy test. In the accuracy comparison, on different problems (items 0–29), Gumbel Soft Thinking and sampling both solve 52 of 60 runs. The 60 runs are two seeds of the same 30 problems, and Gumbel Soft Thinking's two seeds are right on exactly the same problems, so we treat problems, not runs, as the unit. Per seed, sampling solves 27 and 25 of 30 and Gumbel Soft Thinking 26 and 26. Bootstrapped over problems, the intervals are [0.75, 0.97] for sampling and [0.73, 0.97] for Gumbel Soft Thinking, and the mean per-problem difference is 0.00 [−0.10, +0.10]. On seed 0, Gumbel Soft Thinking alone is right on 1 problem and sampling alone on 2 ($p = 1.00$). Vanilla Soft Thinking shows the Greedy Pitfall of Wu et al. [5] in its behaviour: 14 of its 30 runs end in Cold Stop, a repetition loop, and all of its 5 runs without an answer are among them. Gumbel Soft Thinking never hits Cold Stop, and sampling and Gumbel Soft Thinking solve 4 of those 5 problems in every run. Vanilla's accuracy (22 of 30) is lower by 17 points on seed 0 (95% CI [−33, −1]), but the exact McNemar test does not reject equality ($p = 0.062$ against sampling, $p = 0.22$ against Gumbel), so we treat it as borderline.

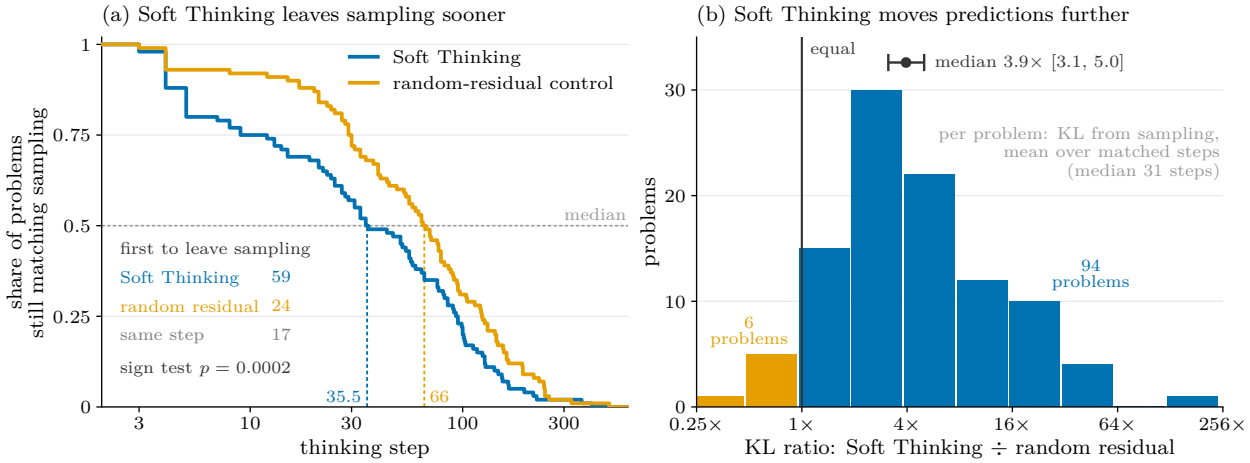


Figure 7. The model reads the residual. Three arms run in lockstep with shared noise on 100 GSM8K problems: sampling (top token only), Soft Thinking (full blend) and a random-residual control (the residual moved to a random token). (a) Share of problems on which each arm has so far picked the same token as sampling at every thinking step. Soft Thinking leaves sampling sooner: median step 35.5 vs 66; it leaves first on 59 problems, the random residual on 24, both at once on 17 (sign test $p = 0.0002$). (b) Per problem, how far Soft Thinking moves next-token predictions away from sampling's, divided by how far the random residual does (KL divergence, averaged over the steps before either arm leaves sampling; median 31 steps). Soft Thinking moves them more on 94 of 100 problems; the median problem is 3.9 times (marker: median and 95% bootstrap CI over problems, [3.1, 5.0]). Items 50–149, not used in any other experiment; seed 0.

Answer. The model reads the residual: it moves next-token predictions a median 3.9 times as much as a random residual of the same size, and it makes Soft Thinking leave sampling sooner. Yet accuracy does not change detectably, and paired by problem, the real residual changes sampling's final answer on the same number of problems as a random one, mostly the same problems. At the token level, the residual is read but, within the power of our test, not used.

5. Discussion

Reconciling with Wu et al. [5]. Wu et al. [5] report that Gumbel Soft Thinking outperforms matched sampling, with the same temperature, top-p and top-k, on three 32B reasoning models. The gain, averaged over their eight benchmarks, is $+0.42$ to $+1.05$ points, with no variance or significance reported; GPQA-Diamond shows the largest gains, up to $+5.05$ points (their Table 2). At 1.5B on GSM8K we find no detectable difference (accuracy comparison: 52 of 60 for both; coupled ablation: 88 vs 86 of 100, McNemar $p = 0.754$). The two findings are compatible. Wu et al. motivate Gumbel-Softmax through the Gumbel-max trick; its consequence is that the top token of Gumbel Soft Thinking is distributed exactly as a sampled token, which we confirm (flip rate 0.45 for both). Dirichlet noise has no such guarantee, and in our runs its top token flips 0.03 less often than a sampled token would. Wu et al. also find Dirichlet no better than sampling, although our Dirichlet uses $\gamma = 1.0$ and theirs 4.0 by default. Any accuracy difference between Gumbel Soft Thinking and sampling must therefore come from the residual. Our coupled ablation holds the noise fixed and removes only the residual. The model reads it, yet accuracy does not change detectably. Our paired interval runs from 5 points below sampling to 9 points above it, a range that includes the gains Wu et al. report. For the Gumbel variant this refines their single-threaded account: the top token is a sample, and the residual is read but leaves accuracy unchanged within our power. Three differences could separate the settings. A 32B model may use a residual that a 1.5B model only stores. GSM8K is near ceiling for our model (86 to 88% in the coupled ablation), while the largest reported gains are on harder benchmarks. Our study lacks the power to resolve effects of a few points, and Wu et al. report no variance. Running the coupled ablation on a 32B model and a harder benchmark would show which explanation applies.

The tension with Rizvi-Martel et al. [6]. They conclude that off-the-shelf models collapse Soft Thinking inputs within the first few layers, while our Fig. 3 shows a gradual collapse that is complete only at the output. The two results measure different things. Their evidence for early collapse comes from vanilla Soft Thinking, every 50 decoding steps: the logit-lens entropy of separate soft and discrete runs, and the KL divergence between forward passes with the soft input and with its top token. Those inputs are close to one token on average (mean mixing-weight entropy 0.10 to 0.18 nats), so little blend is left to lose. We condition on forks, feed exact 50/50 blends of the two candidates, and track the position between them at every layer. On these inputs the blend is still far from a clean switch at layer 7 (snap 0.37 to 0.70). This agrees with their own test on uniform mixtures, where entropy drops only at the last layers, and with Backour [7], who finds that Qwen models accumulate collapse toward the output. Collapse in the next-token readout also does not mean the runner-up is lost: a trace of it persists, mostly in the keys and values of layers 0 to 7, and shifts later predictions toward it (Section 4.2). Our blends are 50/50, likely an upper bound on real forks, where the median residual is 0.12.

What our evidence says about superposition. Our results separate three levels of token-level superposition: a second candidate present in the input, read by the model, and used. The forced 50/50 blends are present in the input, and a trace of the runner-up persists in the cache. The residual of Gumbel Soft Thinking is read: it shifts next-token predictions far more than a random residual. We find no evidence that it is used. For that, the trace would have to shift predictions on the model's own path, where we detect no shift, and the residual would have to change answers, where we detect no accuracy change and the real residual changes answers no more often than a random one. Two limits keep this from being a general verdict on superposition. Our blends are forced, not learned; models trained with latent thoughts from scratch show signs of using superposition [6], and reinforcement learning with soft tokens is now practical [16, 17]. And we test single positions, not whole reasoning paths: a model could in principle keep two lines of reasoning alive across many steps without either one leaving a detectable trace at a single fork. The coupled ablation applies to trained models unchanged: shared noise gives the arms the same top token until their contexts diverge, so the residual is the only source of

divergence.

6. Limitations

We study one model, DeepSeek-R1-Distill-Qwen-1.5B, on one task, GSM8K, mostly with one seed. Larger models or harder tasks may use the residual where this model does not. Accuracy on GSM8K is near ceiling for this model, which leaves little room for the residual to help. The accuracy tests have low power: the coupled ablation has only 6 and 4 discordant problems, and its paired interval allows differences from 5 points below sampling to 9 points above, so our null result is an absence of evidence at this size. Our tests are at the level of single token positions. We do not test whether the model carries whole reasoning paths in superposition across many steps, and every blend we study is a forced superposition built at the input, not a learned one [6]. The cache results (Sections 4.1 and 4.2) use exact 50/50 blends at forks where the model followed the chosen token, the most favourable case for the runner-up. Real Gumbel forks feed a median residual of 0.12, so these results are likely an upper bound on what real runs carry, if readback grows with the residual; we did not measure readback at realistic weights. The readback and snap scores are projections onto the line between two pure states, so they can saturate or pass 1; this is one of two readings of the non-additivity inside layers 0–3. The single-layer patching results are exploratory and were found after looking at the data. The experiments are not independent samples of problems: the fed-blend log uses a subset of the fork probe's problems and overlaps the accuracy comparison. Our Dirichlet noise uses $\gamma = 1.0$, while Wu et al. [5] use 4.0 by default, so our Dirichlet result may not transfer to their setting. Finally, our decoding settings differ from those of Wu et al. [5] (top-k, thinking budget, Cold Stop); they are matched across our own arms, but our absolute accuracies are not comparable with theirs.

7. Conclusion

Gumbel Soft Thinking feeds a sampled token plus a residual. In a 1.5B reasoning model on GSM8K, we tested whether that residual acts as a token-level superposition. A trace of the runner-up persists, mostly in early-layer keys and values, and the model reads the residual, but accuracy does not change detectably and the real residual changes answers no more often than a random one. The superposition is present and read, but we find no evidence that it is used. Testing learned superposition in trained models, and superposition of whole reasoning paths, with the same shared-noise control is the clearest next step.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2201.11903.

[2] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian, "Training large language models to reason in a continuous latent space," in *Conference on Language Modeling (COLM)*, 2025, arXiv:2412.06769.

[3] Z. Zhang, X. He, W. Yan, A. Shen, C. Zhao, S. Wang, Y. Shen, and X. E. Wang, "Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2505.15778.

[4] H. Zhu, S. Hao, Z. Hu, J. Jiao, S. Russell, and Y. Tian, "Reasoning by superposition: A theoretical perspective on chain of continuous thought," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2505.12514.

[5] J. Wu, J. Lu, Z. Ren, G. Hu, Z. Wu, D. Dai, and H. Wu, "LLMs are single-threaded reasoners: Demystifying the working mechanism of soft thinking," in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2508.03440v4.

[6] M. Rizvi-Martel, G. Rabusseau, and M. Mosbach, "The illusion of superposition? A principled analysis of latent thinking in language models," in *Conference on Language Modeling (COLM)*, 2026, arXiv:2604.06374v2.

[7] A. Backour, "The dynamics of continuous mixture collapse in language models," 2026, arXiv:2609.02049v1.

[8] E. J. Gumbel, *Statistical Theory of Extreme Values and Some Practical Applications: A Series of Lectures*, ser. National Bureau of Standards Applied Mathematics Series. Washington, DC: U.S. Government Printing Office, 1954, no. 33.

[9] C. J. Maddison, D. Tarlow, and T. Minka, "A* sampling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, arXiv:1411.0030.

[10] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, arXiv:2501.12948.

[11] D. Guo, D. Yang, H. Zhang *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.

[12] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," 2021, arXiv:2110.14168.

[13] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-softmax," in *International Conference on Learning Representations (ICLR)*, 2017, arXiv:1611.01144.

[14] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *International Conference on Learning Representations (ICLR)*, 2017, arXiv:1611.00712.

[15] W. Kool, H. van Hoof, and M. Welling, "Stochastic beams and where to find them: The Gumbel-top-k trick for sampling sequences without replacement," in *International Conference on Machine Learning (ICML)*, 2019, pp. 3499–3508, arXiv:1903.06059.

[16] N. Butt, A. Kwiatkowski, I. Labiad, J. Kempe, and Y. Ollivier, "Soft tokens, hard truths," 2025, arXiv:2509.19170v2.

[17] Z. Zheng, Y. Gu, W. Liu, Y. W. Teh, and W. S. Lee, "SofT-GRPO: Surpassing discrete-token LLM reinforcement learning via Gumbel-reparameterized soft-thinking policy optimization," 2025, arXiv:2511.06411v2.

[18] J. Deng, L. Pang, Z. Wei, S. Xu, Z. Duan, K. Xu, Y. Song, H. Shen, and X. Cheng, "LLM latent reasoning as chain of superposition," 2025, arXiv:2510.15522v2.

[19] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2202.05262.

[20] Z. Li, X. Bai, K. Chen, Y. Li, J. Yang, C. Lin, and M. Zhang, "Dynamics within latent chain-of-thought: An empirical study of causal structure," in *International Conference on Machine Learning (ICML)*, 2026, arXiv:2602.08783v3.

[21] Y. Zhang, B. Tang, T. Ju, S. Duan, and G. Liu, "Do latent tokens think? A causal and adversarial analysis of chain-of-continuous-thought," 2025, arXiv:2512.21711v1.

[22] D. Aswal, T. P. Ferraz, Y. Zhou, and M. Peyrard, "Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models," 2026, arXiv:2606.12689v2.

[23] H. Jin, S. Yang, and D. Wang, "Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories," 2026, arXiv:2607.06648v2.

[24] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen *et al.*, "Toy models of superposition," *Transformer Circuits Thread*, 2022, arXiv:2209.10652.

[25] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[26] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[27] R. G. Newcombe, "Improved confidence intervals for the difference between binomial proportions based on paired data," *Statistics in Medicine*, vol. 17, no. 22, pp. 2635–2650, 1998.

A. Pre-registration record

We committed predictions with numeric thresholds to our git repository before each run. Git dates are set by the local machine, so they show the order of commits, not an external timestamp; the Kaggle notebook version times give a second record. Table 2 lists every prediction as written and its outcome, including the ones that failed.

Fork probe (P1–P6) was committed on 7 October 2026 (commit `7d41cd4`), before any of its results. The thresholds were set on a 10-item pilot (items 0–9) and tested on 40 held-out items (10–49). The pilot's raw outputs were not saved, so we report no pilot numbers. P1 was written as "snap ≥ 0.6" before we rescaled snap so that a perfect switch scores 1; on the rescaled measure the same threshold is 0.66, and P1 is met on both scales.

Fed-blend log (P1–P3) and **coupled ablation** (E1, E2) were committed on 10 October 2026 (commit `d2e770f`), before either experiment was run. After writing P1–P3 but before running the fed-blend log, we derived the distribution of the fed top weight in closed form. It showed that a 50/50 fork gives a fed top weight of at least 0.9 with probability 0.50 under Gumbel noise, and a 75/25 fork with probability 0.60, so P1 and P2 could not reach their 70% thresholds. We kept P1 and P2 as written and added A1–A3 in the same commit, again before running.

Two predictions failed in an informative way. Fed-blend log P3 expected near one-token blends at 90% of all thinking steps; Gumbel reached 85%. E1 expected the residual to move predictions less than a random residual, as it would if the model ignored the residual; the result went the other way (Section 4.4). E1 was written for the KL averaged over each arm's own steps up to and including its first divergence, and the table reports it on that measure: Soft Thinking moved predictions less on 14 of 100 problems (mean KL 0.545 against 0.009). In the main text we report the KL over steps matched across arms, because the registered measure is dominated by the divergence step itself; on matched steps Soft Thinking moved predictions less on 6 of 100. The verdict is the same under both measures. One summary statistic was also chosen after seeing the data: we report the median per-problem KL ratio (3.9) because the ratio of means (5.7) turned out to be driven by five problems (Section 4.4).

B. Robustness of the storage results

The tables and figures in this appendix back Figs. 4 and 5. Table 3 recomputes the pooled patching shares when only forks with a gap of at least 0.1 are kept, when each problem is dropped in turn, and as a median over forks. Table 4 gives the number vs number single layers that stand out, with Bonferroni-corrected lower bounds over all 56 single-layer tests. Table 5 reports the number vs word blocks, whose gap is too small to localise. Figure 8 shows the readback at single positions of the forced text, and Fig. 9 the per-fork spread behind the pooled shares.

C. Embedding-norm check for the random-residual control

The random-residual control of the coupled ablation moves the residual onto a random vocabulary row. If random rows had much smaller norms than the tokens the model reads while reasoning, the control would carry a weaker signal than Soft Thinking, and the KL comparison of Section 4.4 would be unfair to it. We regenerated the 8,192 random rows that the coupled ablation drew, with the same keyed generator, and compared their norms with all real-token rows and with 9,168 tokens from sampled reasoning traces on 5 problems (items 50–54). The median norm is 1.195 for the random rows, 1.195 for all real-token rows, 1.196 when real rows are weighted by usage, and 1.202 for reasoning tokens. The ratio of random rows to reasoning tokens is 0.994 at the median and 0.961 at the mean. The random rows have a heavier low tail (10th percentile 0.897 against 1.058), so a minority of draws carries a weaker signal. This cannot explain a median KL ratio of 3.9 between Soft Thinking and the control. These values come from the saved output of a separate GPU run (`neuralese_exp23_norms.py`); the check scripts compare against that output rather than recompute it from the model.

D. Reproducibility

Model and software. Every experiment loads `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B` from the Hugging Face Hub at revision `ad9f0ae0864d7fbcd1cd905e3c6c5b069cc8b562`, last modified on 24 February 2025, before any of our runs. The scripts load the Hub identifier, so all runs used this revision. Each Kaggle notebook installed the latest `transformers`; the install logs do not print versions, but the downloaded wheel identifies them. The fork probe and the coupled ablation used `transformers` 5.19.0, and the accuracy comparison and the fed-blend log used 5.20.1 (inferred from the wheel size and upload dates). The saved outputs of the accuracy comparison and the fed-blend log come from a later rerun that reproduced the originally logged numbers exactly. The embedding-norm check (Appendix C) recorded its full environment: torch 2.11.0+cu128, `transformers` 5.20.2, CUDA 12.8 and Python 3.13.15 on an NVIDIA Tesla T4. All runs used Python 3.13; torch and CUDA versions are recorded only for the norm check.

Code, outputs and checks. Table 6 maps each experiment to its script and saved output. The logs of the Kaggle runs that produced these outputs are in `kaggle_logs/`. Two scripts recom-

Table 2. Pre-registered predictions, as written, with their outcomes. Fork probe: GSM8K items 10–49; fed-blend log: items 10–29; coupled ablation: items 50–149. nn = number vs number, ww = word vs word.

	Prediction as written	Observed	Met
Fork probe: collapse, cache trace, location (commit `7d41cd4`)			
P1	nn final-layer snap ≥ 0.6 (0.66 rescaled)	0.84	yes
P2	50/50 input: next token is pure A's or B's at $\geq 85\%$ of nn forks	95%	yes
P3	Runner-up's text: readback CI lower bound > 0, nn and ww	0.35, 0.27	yes
P4	Own text: $\lvert\text{readback}\rvert < 0.1$, nn and ww	0.01, 0.02	yes
P5	Layers 0–7 close $\geq 50\%$ of the gap (remove, insert; nn, ww)	smallest 0.71	yes
P6	Layers 0–7 minus 12–27: CI lower bound > 0 (same four cases)	smallest 0.31	yes
Fed-blend log (commit `d2e770f`)			
P1	Gumbel: fed top weight ≥ 0.9 at $\geq 70\%$ of forks	47%	no
P2	Dirichlet: same, $\geq 70\%$ of forks	37%	no
P3	Gumbel: fed top weight ≥ 0.9 at $\geq 90\%$ of all thinking steps	85%	no
A1	Gumbel: fed top weight ≥ 0.9 at 35–65% of forks	47%	yes
A2	Dirichlet: fed top weight ≥ 0.9 at 25–60% of forks	37%	yes
A3	Gumbel flip rate within 0.05 of the sampling rate	0.45 vs 0.45	yes
Coupled ablation (commit `d2e770f`)			
E1	Soft Thinking moves predictions less than the random residual on most problems	14 of 100[a]	no
E2	Soft Thinking vs sampling: McNemar $p > 0.05$ and accuracy difference ≤ 5 points	$p = 0.754$, 2 points	yes

[a] As registered: KL over each arm's own steps; 6 of 100 on matched steps (Section 4.4).

Table 3. Robustness of the patching shares in Figure 5. Pooled: share of the summed blend–control gap closed (as plotted). Gap ≥ 0.1: the same, keeping only forks whose gap is at least 0.1. Leave one out: range of the pooled share when each problem is dropped in turn. Median: median of per-fork shares over forks with gap ≥ 0.1. Fork probe, GSM8K items 10–49, 50/50 blends, forks where the model followed the chosen token.

	Remove				Insert			
Patched	Pooled	Gap ≥ 0.1	Leave one out	Median	Pooled	Gap ≥ 0.1	Leave one out	Median
Number vs number: 96 forks, 78 with gap ≥ 0.1								
layers 0–3	0.58	0.56	0.56–0.62	0.53	0.55	0.54	0.54–0.59	0.49
layers 4–7	0.19	0.19	0.18–0.20	0.16	0.16	0.16	0.14–0.17	0.10
layers 8–11	0.06	0.06	0.05–0.06	0.02	0.09	0.09	0.08–0.10	0.04
layers 12–15	0.12	0.10	0.11–0.12	0.06	0.17	0.16	0.15–0.18	0.10
layer 0	0.38	0.36	0.35–0.41	0.12	0.16	0.16	0.14–0.17	0.02
layer 1	0.21	0.20	0.19–0.22	0.14	0.07	0.07	0.06–0.08	0.05
layer 2	0.20	0.19	0.19–0.22	0.12	0.05	0.06	0.05–0.06	0.03
layer 7	0.13	0.13	0.12–0.13	0.11	0.12	0.12	0.11–0.13	0.06
layer 13	0.08	0.07	0.07–0.09	0.04	0.13	0.12	0.12–0.14	0.05
Word vs word: 139 forks, 108 with gap ≥ 0.1								
layers 0–3	0.66	0.63	0.65–0.69	0.66	0.75	0.72	0.73–0.77	0.78
layers 4–7	0.10	0.10	0.10–0.11	0.09	0.16	0.16	0.15–0.17	0.15
layers 8–11	0.04	0.05	0.04–0.04	0.03	0.07	0.07	0.06–0.07	0.06
layers 12–15	0.03	0.04	0.03–0.04	0.02	0.06	0.06	0.05–0.06	0.05

pute every number in the paper from the saved outputs and stop on any mismatch: `verify_claims.py` checks the claim ledger, and `verify_robustness.py` checks every cell of Table 1 and Tables 3–5. A third script, `paper/check_numbers.py`, checks that every number in the text matches the ledger row named next to it in the LaTeX source. Figures are drawn by one script each (`figures/make_*.py`) from the saved outputs.

E. Use of AI assistants

The author used an AI assistant (Anthropic's Claude) to summarise related literature, to draft Python scripts for the experiments, analyses, figures and tables, and to summarise results and organise experimental data. The author designed and ran all experiments, reviewed and tested all code, checked every reported number against the saved outputs and every citation against its source, and takes full responsibility for the content of this paper.

Table 4. Number vs number single layers that stand out in Figure 5c. 95% CI: bootstrap over problems (2,000 resamples). Corrected lower: lower bound of the Bonferroni-corrected interval for all 56 single-layer tests (28 layers × remove/insert; 20,000 resamples). Every bound is above 0. The single-layer analysis was not pre-registered.

	Remove			Insert		
Patched	Share	95% CI	Corrected lower	Share	95% CI	Corrected lower
layer 0	0.38	[0.27, 0.50]	0.20	0.16	[0.10, 0.23]	0.06
layer 1	0.21	[0.15, 0.27]	0.12	0.07	[0.03, 0.10]	0.01
layer 2	0.20	[0.15, 0.26]	0.12	0.05	[0.03, 0.08]	0.01
layer 7	0.13	[0.10, 0.16]	0.08	0.12	[0.08, 0.17]	0.05
layer 13	0.08	[0.05, 0.12]	0.04	0.13	[0.09, 0.17]	0.06

Table 5. Number vs word forks (34 forks from 21 problems), not shown in Figure 5. The mean blend–control gap is only 0.11, so the shares are unstable (wide CIs, values above 1 and below 0) and cannot localise the runner-up. 95% CI: bootstrap over problems.

	Remove		Insert	
Patched	Share	95% CI	Share	95% CI
layers 0–3	0.98	[0.63, 2.31]	0.90	[0.67, 1.55]
layers 4–7	0.11	[0.02, 0.59]	−0.01	[−0.24, 0.11]
layers 8–11	−0.01	[−0.10, 0.02]	0.00	[−0.09, 0.06]
layers 12–15	−0.03	[−0.12, 0.01]	−0.02	[−0.25, 0.05]
layers 16–19	0.14	[0.00, 0.38]	0.08	[−0.03, 0.21]
layers 20–23	0.07	[−0.01, 0.20]	−0.01	[−0.11, 0.05]
layers 24–27	0.04	[−0.14, 0.12]	0.03	[−0.09, 0.10]

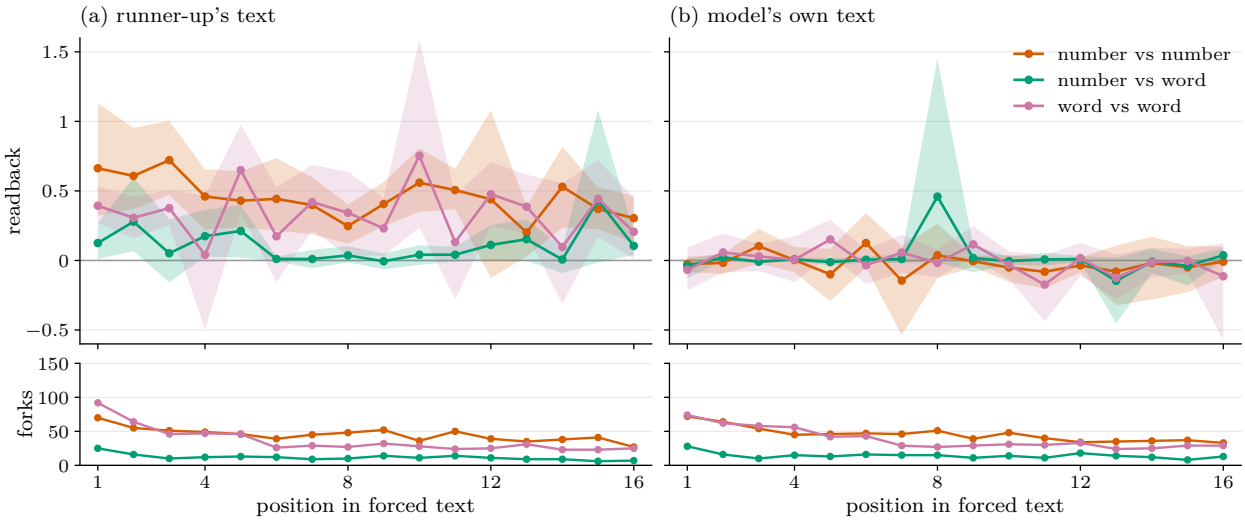


Figure 8. Readback at each position of the forced text, without the binning of Fig. 4b–c. Top: mean readback (control minus blend) with 95% bootstrap CIs over problems, for the runner-up's text (left) and the model's own text (right). Bottom: number of forks with a defined score at that position (positions where the chosen token and the runner-up barely differ are skipped). With the runner-up's text, the mean stays above 0 at every position for number vs number and word vs word; with the model's own text it stays around 0. Single positions hold few forks, so their CIs are wide; this is why Fig. 4 pools positions into log-spaced bins.

Table 6. Experiments, scripts and saved outputs (under `results/`; seed 0 unless stated).

Experiment	Question	Items	Script	Saved output
Accuracy comparison	Accuracy	0–29	`neuralese_eval.py`	`exp16/gsm8k_eval.jsonl`
Fork probe	Q1 snap	10–49	`neuralese_sweep.py`	`exp21/step4_alpha_sweep_*.jsonl`
Fork probe	Q2 readback	10–49	`neuralese_kv_carryover.py`	`exp21/step5_kv_carryover_*.jsonl`
Fork probe	Q2 patching	10–49	`neuralese_kv_patch.py`	`exp21/step5_kv_patch_*.jsonl`
Fed-blend log	Q3 fed blends	10–29	`neuralese_mixtures.py`	`exp22/exp22_mixtures_*.jsonl`
Coupled ablation	Q4 ablation	50–149	`neuralese_residual_ablation.py`	`exp23/exp23_residual_ablation_*.jsonl`
Coupled ablation	Norm check	50–54	`neuralese_exp23_norms.py`	`exp23/exp23_embedding_norms.json`

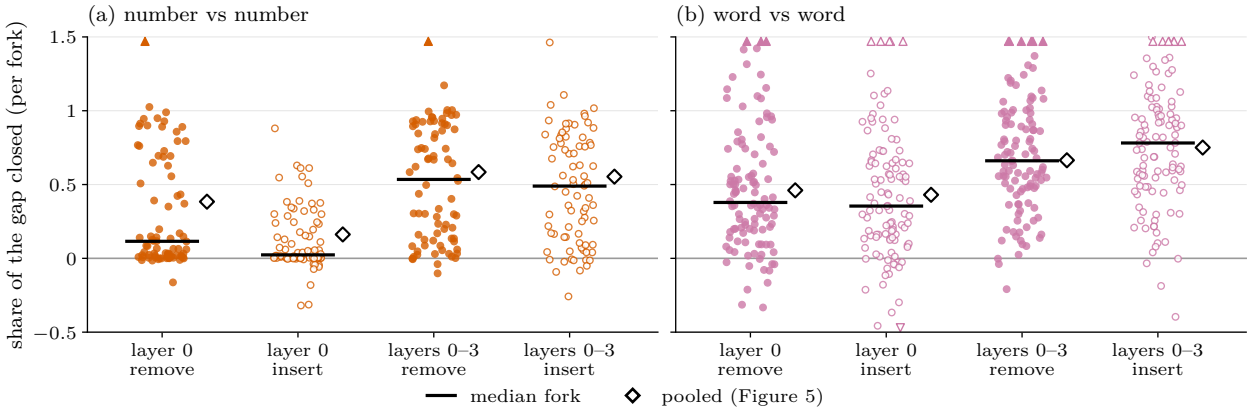


Figure 9. Per-fork shares behind the pooled bars of Fig. 5, for layer 0 and layers 0–3. Each dot is one fork with a blend–control gap of at least 0.1 (78 number vs number forks, 108 word vs word); filled = remove, open = insert. Black bar: median per fork. Diamond: pooled share, as plotted in Fig. 5. For number vs number, layer 0 is uneven across forks: half of them have a remove share below 0.12 and a quarter are above 0.68, so the pooled 0.38 sits well above the median fork (0.12; insert: 0.16 vs 0.02). For layers 0–3 the median fork is close to the pooled share. Dots outside the axis range are drawn at its edge as triangles.